

Experiment 1: Insertion Sort – Live Leaderboard Updates

Aim

To implement the **Insertion Sort** algorithm in Python to update a player's score in a leaderboard without sorting the entire list again.

Problem Statement

An online game updates a leaderboard whenever a player's score changes. Instead of sorting the complete leaderboard again, use **Insertion Sort** to insert the updated score into its correct position.

Algorithm

Step 1: Start.

Step 2: Read the number of scores.

Step 3: Read the leaderboard scores (in descending order).

Step 4: Read the updated score.

Step 5: Add the updated score to the list.

Step 6: Apply Insertion Sort to arrange the scores in descending order.

Step 7: Display the updated leaderboard.

Step 8: Stop.

Source Code (Python)

```
main.py
1 # Live Leaderboard Updates using Insertion Sort
2
3 n = int(input("Enter number of scores: "))
4 board = []
5
6 print("Enter leaderboard scores:")
7 for i in range(n):
8     board.append(int(input()))
9
10 new_score = int(input("Enter updated score: "))
11
12 board.append(new_score)
13
14 for i in range(1, len(board)):
15     key = board[i]
16     j = i - 1
17
18     while j >= 0 and board[j] < key:
19         board[j + 1] = board[j]
20         j -= 1
21
22     board[j + 1] = key
23
24 print("Updated Leaderboard:", board)
```

Output

```
Output
Enter number of scores: 5
Enter leaderboard scores:
980
875
760
690
500
Enter updated score: 820
Updated Leaderboard: [980, 875, 820, 760, 690, 500]

=== Code Execution Successful ===
```

Complexity Analysis

Aspect	Complexity
Time Complexity	$O(n)$ (for inserting one score)
Space Complexity	$O(1)$

Result

The Insertion Sort algorithm was successfully implemented to update the leaderboard. The new score was inserted into the correct position without re-sorting the entire list.